



docker

دليل تعليمي مختصر ومنظم

Amine Mohamed

Telegram:@codeDarija

جدول المحتويات

1. مقدمة إلى Docker ومفاهيمه الأساسية — ما هو Docker ولماذا نستخدمه؟

2. مقدمة إلى Docker ومفاهيمه الأساسية — مقارنة بين Docker والآلات الافتراضية (Virtual Machines)

3. مقدمة إلى Docker ومفاهيمه الأساسية — المكونات الأساسية في Docker: الصور (Images)، الحاويات (Containers)، السجلات (Registries)

4. مقدمة إلى Docker ومفاهيمه الأساسية — فهم البنية الكلية لـ Docker

5. تثبيت Docker وإعداده — متطلبات النظام لتثبيت Docker

6. تثبيت Docker وإعداده — تثبيت Docker Desktop على أنظمة التشغيل Windows و macOS

7. تثبيت Docker وإعداده — تثبيت Docker Engine على أنظمة Linux

8. تثبيت Docker وإعداده — التحقق من صحة التثبيت وتشغيل أول أمر Docker

9. التعامل مع صور Docker (Images) — مفهوم صور Docker ودورها

10. التعامل مع صور Docker (Images) — سحب الصور من Docker Hub باستخدام الأمر `docker pull`

11. التعامل مع صور Docker (Images) — عرض الصور المتوفرة محلياً وفحص تفاصيلها

12. التعامل مع صور Docker (Images) — حذف الصور غير المستخدمة من النظام

13. تشغيل وإدارة حاويات Docker (Containers) — تشغيل حاوية Docker الأولى باستخدام الأمر `docker run`

14. تشغيل وإدارة حاويات Docker (Containers) — دورة حياة الحاوية: الإنشاء، البدء، الإيقاف، الإزالة

15. تشغيل وإدارة حاويات Docker (Containers) — عرض
الحاويات قيد التشغيل والموقوفة

16. تشغيل وإدارة حاويات Docker (Containers) — الوصول
إلى صدف الحاوية (Container Shell) وتنفيذ الأوامر

17. تشغيل وإدارة حاويات Docker (Containers) — ربط
المنافذ (Port Mapping) للسماح بالوصول الخارجي للحاويات

18. تشغيل وإدارة حاويات Docker (Containers) — فحص
تفاصيل الحاوية وسجلاتها

19. بناء صور Docker مخصصة باستخدام Dockerfile — ما
هو Dockerfile وكيف يعمل؟

20. بناء صور Docker مخصصة باستخدام Dockerfile —
تعليمات Dockerfile الأساسية: FROM, RUN, CMD,
ENTRYPOINT, COPY, ADD

21. بناء صور Docker مخصصة باستخدام Dockerfile — بناء
صورة Docker من Dockerfile باستخدام الأمر `docker build`

22. بناء صور Docker مخصصة باستخدام Dockerfile —
أفضل الممارسات لكتابة Dockerfile فعال وآمن

23. تخزين البيانات في Docker (Volumes) — فهم تحديات استمرارية البيانات في الحاويات

24. تخزين البيانات في Docker (Volumes) — مفهوم الـ Volumes وأنواعها (Bind Mounts, Docker Volumes)

25. تخزين البيانات في Docker (Volumes) — استخدام Bind Mounts لربط المجلدات من المضيف

26. تخزين البيانات في Docker (Volumes) — إنشاء وإدارة Docker Volumes لتخزين البيانات الدائمة

27. تخزين البيانات في Docker (Volumes) — أوامر إدارة الـ Volumes (إنشاء، عرض، حذف)

28. شبكات Docker (Networking) — نظرة عامة على شبكات Docker وأنواعها

29. شبكات Docker (Networking) — شبكة الجسر الافتراضية (Default Bridge Network)

30. شبكات Docker (Networking) — إنشاء شبكات جسر مخصصة (Custom Bridge Networks)

31. شبكات (Networking) Docker — ربط الحاويات بالشبكات المختلفة

32. شبكات (Networking) Docker — اكتشاف الحاويات (Container Discovery) داخل الشبكات

33. إدارة التطبيقات متعددة الحاويات بـ Docker Compose — مقدمة إلى Docker Compose ودوره في إدارة التطبيقات

34. إدارة التطبيقات متعددة الحاويات بـ Docker Compose — هيكل ملف `docker-compose.yml` وتعريف الخدمات (Services)

35. إدارة التطبيقات متعددة الحاويات بـ Docker Compose — تعريف الشبكات والـ Volumes في ملف Compose

36. إدارة التطبيقات متعددة الحاويات بـ Docker Compose — تشغيل التطبيقات متعددة الحاويات باستخدام `docker-compose up`

37. إدارة التطبيقات متعددة الحاويات بـ Docker Compose — إيقاف وإزالة تطبيقات Compose

38. Docker Hub وسجلات الصور (Registries) — ما هو Docker Hub ولماذا نستخدمه؟

39. Docker Hub وسجلات الصور (Registries) — إنشاء حساب على Docker Hub

40. Docker Hub وسجلات الصور (Registries) — وسم الصور (Tagging Images) لرفعها للسجل

41. Docker Hub وسجلات الصور (Registries) — دفع الصور إلى Docker Hub باستخدام الأمر `docker push`

42. Docker Hub وسجلات الصور (Registries) — استخدام السجلات الخاصة (Private Registries)

43. مفاهيم متقدمة ونصائح عملية — البناء متعدد المراحل (Multi-stage Builds) في Dockerfile

44. مفاهيم متقدمة ونصائح عملية — أساسيات أمان الحاويات وتجنب الثغرات الأمنية

45. مفاهيم متقدمة ونصائح عملية — تحديد حدود الموارد للحاويات (CPU, Memory)

46. مفاهيم متقدمة ونصائح عملية — تنظيف كائنات Docker غير المستخدمة (Pruning)

47. مفاهيم متقدمة ونصائح عملية — استكشاف أخطاء
Docker الشائعة وإصلاحها

مقدمة إلى Docker ومفاهيمه

الأساسية — ما هو Docker ولماذا نستخدمه؟

ما هو Docker ولماذا نستخدمه؟

في هذا الدرس، سنتعرف على Docker، الأداة الثورية التي غيرت طريقة بناء ونشر وتشغيل التطبيقات. سنفهم ما هو Docker ولماذا أصبح لا غنى عنه في عالم البرمجة الحديث.

ما هو Docker؟

Docker هو منصة تمكّنك من تطوير ونشر وتشغيل التطبيقات في بيئات معزولة تسمى **حاويات** (Containers). تخيل الحاوية كصندوق صغير يحتوي على كل ما يحتاجه تطبيقك ليعمل: الكود، وقت التشغيل، المكتبات، وملفات الإعدادات. هذا يضمن أن التطبيق يعمل بنفس الطريقة في أي مكان، بغض النظر عن اختلاف بيئة التشغيل الأساسية.

مثال عملي

```
docker run hello-world
```

شرح الكود

يستخدم هذا الأمر لتشغيل حاوية جديدة. `docker run` يخبر Docker بتشغيل تطبيق. `hello-world` هو اسم الصورة التي سيتم استخدامها، وهي صورة بسيطة تعرض رسالة ترحيب لتأكيد أن Docker يعمل بشكل صحيح على جهازك، دون الحاجة إلى تثبيت أي شيء آخر.

- يستخدم Docker الحاويات لعزل التطبيقات وبيئاتها عن النظام الأساسي.
- يضمن تشغيل التطبيق بنفس الشكل تمامًا في جميع البيئات (تطوير، اختبار، إنتاج).
- يسهل نشر التطبيقات ويقلل بشكل كبير من مشاكل التوافقية بين البيئات المختلفة.

مقدمة إلى Docker ومفاهيمه الأساسية — مقارنة بين Docker والآلات الافتراضية (Virtual Machines)

مقارنة بين Docker والآلات الافتراضية (Virtual Machines)

في هذا الدرس، سنستكشف الفروقات الأساسية بين **Docker** الذي يعتمد على الحاويات (Containers) والآلات الافتراضية (**Virtual Machines - VMs**)، لفهم متى وكيف نستخدم كل منهما بكفاءة في تطوير ونشر التطبيقات.

الفروقات الجوهرية

تُقدم الآلات الافتراضية محاكاة كاملة لجهاز حاسوب، حيث تمتلك كل VM نظام تشغيل خاص بها وتستهلك موارد كبيرة. أما Docker، فيستخدم الحاويات التي تشارك نواة نظام التشغيل المضيف، مما يجعلها أخف وأسرع بكثير في التشغيل وأقل استهلاكاً للموارد.

مثال توضيحي (تشغيل حاوية Docker)

```
docker run -d -p 80:80 --name mynginx nginx
```

شرح الكود

هذا الأمر يُظهر مدى سهولة وسرعة تشغيل خدمة ويب باستخدام Docker. يقوم الكود بتشغيل حاوية جديدة مبنية على صورة `nginx`، تعمل في الخلفية (`-d`)، وتوجه حركة المرور من المنفذ 80 في جهازك إلى المنفذ 80 داخل الحاوية (`-p 80:80`)، مع تسمية هذه الحاوية باسم `.mynginx`.

- **كفاءة الموارد:** Docker يستهلك موارد أقل بكثير من الآلات الافتراضية لأنه لا يحتاج لنظام تشغيل كامل لكل حاوية.
- **السرعة:** الحاويات تبدأ وتتوقف بشكل أسرع بكثير من الآلات الافتراضية.
- **العزل:** الآلات الافتراضية توفر عزلاً أقوى وأشمل، بينما الحاويات تعزل التطبيقات في بيئة مستقلة لكنها تشارك نواة نظام التشغيل.

مقدمة إلى Docker ومفاهيمه الأساسية — المكونات الأساسية في Docker: الصور (Images)، الحاويات (Containers)، السجلات (Registries)

المكونات الأساسية في Docker: الصور (Images)، الحاويات (Containers)، السجلات (Registries)

تُعَدُّ هذه المكونات حجر الأساس في فهم كيفية عمل Docker. سنتعرف هنا على الصور والحاويات والسجلات، وكيف تتفاعل معًا لتوفير بيئة فعالة لتطوير ونشر التطبيقات.

الصور (Images)

هي قوالب للقراءة فقط تحتوي على تعليمات كاملة لبناء بيئة تطبيق معينة. تشمل الكود، المكتبات، التبعيات، وأي ملفات تكوين يحتاجها التطبيق ليعمل. فكر فيها كـ "مخطط" أو "خريطة طريق" لتطبيقك.

الحاويات (Containers)

الحاويات هي نُسخ قابلة للتشغيل (instances) من الصور. إنها بيئات معزولة وخفيفة الوزن تعمل فيها التطبيقات بشكل مستقل عن النظام المضيف والتطبيقات الأخرى، مما يضمن الاتساق عبر البيئات المختلفة.

السجلات (Registries)

هي مستودعات مركزية لتخزين صور Docker ومشاركتها. Docker Hub هو أشهر سجل عام يتيح لك سحب الصور الجاهزة أو دفع صورك الخاصة لمشاركتها مع الآخرين أو للاستخدام المستقبلي.

مثال عملي

```
docker run hello-world
```

شرح الكود

هذا الأمر يقوم بتشغيل حاوية جديدة. إذا لم تكن صورة **hello-world** موجودة محليًا على جهازك، فسيقوم Docker بسحبها تلقائيًا من سجل Docker Hub. بعد ذلك، يتم إنشاء حاوية من هذه الصورة وتشغيلها، لتعرض رسالة ترحيب بسيطة.

- **الصور (Images):** قوالب لتطبيقاتك، تحتوي على كل ما تحتاجه للتشغيل.

- **الحاويات (Containers):** تُسخ قيد التشغيل ومعزولة من هذه الصور، حيث يعمل تطبيقك فعليًا.
- **السجلات (Registries):** مستودعات لتخزين ومشاركة الصور، مثل Docker Hub.

مقدمة إلى Docker ومفاهيمه

الأساسية — فهم البنية

الكلية لـ Docker

فهم البنية الكلية لـ Docker

لفهم كيفية عمل Docker، من الضروري التعرف على بنيته الأساسية. يعتمد Docker على نموذج قوي يوزع المهام بين مكونات مختلفة لضمان المرونة والكفاءة في إدارة التطبيقات.

مكونات بنية Docker

يعمل Docker بنموذج العميل-الخادم (Client-Server). يتألف من محرك Docker (Daemon)، وهو الخادم الذي يقوم بالعمل الفعلي، وعميل Docker (CLI)، وهو الواجهة التي يتفاعل بها المستخدمون. بالإضافة إلى ذلك، يوجد سجل Docker (Registry) لتخزين ومشاركة صور Docker.

مثال على الكود

```
docker run hello-world
```

شرح الكود

هذا الأمر يطلب من عميل Docker تشغيل حاوية من الصورة المسماة "hello-world". إذا لم تكن الصورة موجودة محليًا، فسيقوم محرك Docker (ال Daemon) بسحبها من Docker Hub (السجل الافتراضي)، ثم ينشئ حاوية جديدة ويقوم بتشغيلها.

- يعتمد Docker على بنية العميل-الخادم (Client-Server) لإدارة المهام.
- المكونات الأساسية هي: محرك Docker (Daemon)، عميل Docker (CLI)، وسجل Docker (Registry).
- محرك Docker هو المسؤول عن بناء وتشغيل وإدارة الحاويات والصور.

تثبيت Docker وإعداده — متطلبات النظام لتثبيت Docker

متطلبات النظام لتثبيت Docker

يعد فهم متطلبات النظام خطوة أساسية قبل تثبيت Docker لضمان عملية تثبيت سلسلة وتشغيل فعال. سنتعرف هنا على أبرز هذه المتطلبات التي تختلف باختلاف نظام التشغيل.

ما هي متطلبات نظام Docker؟

يتطلب Docker نظام تشغيل 64-bit (مثل Windows 10، macOS 10.15، Linux حديثة)، ومعالجًا يدعم المحاكاة الافتراضية، وذاكرة وصول عشوائي (RAM) لا تقل عن 4GB، ومساحة قرص صلب كافية.

مثال على التحقق من النظام

```
uname -a
```

شرح الكود

يستخدم هذا الأمر في أنظمة Linux/macOS لعرض معلومات مفصلة عن نواة نظام التشغيل، بما في ذلك نوع النظام، إصدار النواة، وتاريخ الإنشاء. يساعد هذا في التأكد من أن نظامك يلبي متطلبات Docker الأساسية من حيث نوع نظام التشغيل ومعماريته.

- **نظام تشغيل Docker-64 bit:** يتطلب نظام تشغيل bit-64 مثل macOS, Windows 10 Pro/Enterprise/Education, أو Linux.
- **ذاكرة ومعالج كافيان:** يوصى بحد أدنى 4GB من الذاكرة العشوائية ومعالج يدعم المحاكاة الافتراضية لتشغيل Docker بكفاءة.
- **مساحة تخزين:** توفير مساحة كافية على القرص الصلب أمر ضروري لحفظ الصور (Images) والحاويات (Containers) الخاصة بـ Docker.

تثبيت Docker وإعداده — تثبيت Docker Desktop على أنظمة التشغيل Windows و macOS

تثبيت Docker Desktop على أنظمة التشغيل Windows و macOS

يتناول هذا الدرس خطوات تثبيت **Docker Desktop**، الأداة الأساسية لتشغيل Docker على أنظمة التشغيل Windows و macOS، مما يتيح لك بناء وتشغيل التطبيقات المعبأة (contained applications) بسهولة على جهازك المحلي.

المفهوم الأساسي

Docker Desktop هو تطبيق يجمع بين محرك Docker و Docker CLI و Docker Compose، بالإضافة إلى واجهة مستخدم رسومية (GUI). يوفر بيئة كاملة ومتكاملة لتطوير وتشغيل التطبيقات المعبأة (containers) على أنظمة Windows و macOS، مما يلغي الحاجة إلى إعداد بيئات افتراضية معقدة يدوياً.

مثال عملي

بعد إتمام عملية التثبيت الرسومية (بالنقر على "Next" ثم "Finish")، يمكنك التحقق من عمل Docker باستخدام أمر بسيط في الطرفية (Command Prompt أو Terminal):

```
docker run hello-world
```

شرح الكود

بعد تثبيت Docker Desktop بنجاح، يمكنك استخدام هذا الأمر للتحقق من أن Docker يعمل بشكل صحيح. يقوم الأمر `docker run` بتنزيل صورة "hello-world" من Docker Hub إذا لم تكن موجودة محلياً، ثم ينشئ حاوية (container) من هذه الصورة ويشغلها. تعرض الحاوية رسالة تأكيد بسيطة في الطرفية، مما يدل على أن Docker جاهز للاستخدام.

- **Docker Desktop** هو الطريقة الموصى بها لتشغيل Docker على Windows و macOS.
- يتضمن Docker Desktop جميع الأدوات والمكونات اللازمة لـ Docker في حزمة واحدة.
- يمكن التحقق من نجاح التثبيت بتشغيل الأمر `docker run hello-world` في الطرفية.

تثبيت Docker وإعداده — تثبيت Docker Engine على أنظمة Linux

تثبيت Docker Engine على أنظمة Linux

في هذا الدرس، سنتعلم كيفية تثبيت **Docker Engine**، وهو المكون الأساسي لتشغيل دوكر، على أشهر توزيعات أنظمة Linux. يعتبر تثبيت دوكر الخطوة الأولى والضرورية للبدء في استخدام الحاويات وبناء تطبيقاتك المعبأة.

الخطوات الأساسية لتثبيت Docker Engine

لتثبيت Docker Engine على نظام Linux، ستحتاج عادةً إلى تحديث قوائم حزم نظام التشغيل، ثم إضافة مستودع دوكر الرسمي، وأخيرًا تثبيت الحزم الرئيسية لـ Docker Engine باستخدام مدير الحزم الخاص بنظامك (مثل **apt** في Debian/Ubuntu أو **yum/dnf** في CentOS/Fedora).

مثال على أوامر التثبيت والتحقق (Debian/Ubuntu)

تحديث قائمة الحزم وتثبيت دوكر (بعد إعداد المستودع) #
`sudo apt update`

```
sudo apt install docker-ce docker-ce-cli containerd.io
```

التحقق من تثبيت دوكر بتشغيل حاوية تجريبية

```
sudo docker run hello-world
```

شرح الكود

يقوم الأمر الأول `sudo apt update` بتحديث قائمة حزم النظام لضمان الحصول على أحدث معلومات المستودعات. ثم يقوم الأمر `sudo apt install docker-ce docker-ce-cli containerd.io` بتثبيت المكونات الأساسية لـ **Docker Engine**. أما الأمر الأخير `sudo docker run hello-world`، فيقوم بتنزيل وتشغيل حاوية تجريبية صغيرة تُظهر رسالة "Hello from Docker"، مما يؤكد أن دوكر يعمل بنجاح على نظامك.

- **Docker Engine** هو أساس تشغيل بيئة دوكر ويجب تثبيته أولاً على نظام Linux.
- تتضمن عملية التثبيت الشائعة إضافة مستودع دوكر الرسمي ومن ثم استخدام مدير الحزم المناسب لنظامك.
- يمكنك التحقق من نجاح التثبيت بتشغيل حاوية "hello-world" التجريبية التي تعرض رسالة تأكيد.

تثبيت Docker وإعداده — التحقق من صحة التثبيت وتشغيل أول أمر Docker

التحقق من صحة التثبيت وتشغيل أول أمر Docker

بعد إتمام عملية تثبيت Docker، من الضروري التأكد من أنه يعمل بشكل صحيح على نظامك. في هذا الدرس، سنتعلم كيفية التحقق من صحة التثبيت وتشغيل أول حاوية Docker لك لتأكيد جاهزيته للعمل.

التحقق من التثبيت

للتأكد من أن Docker قد تم تثبيته بنجاح ويعمل بشكل صحيح، يجب عليك التحقق من إصداره ومن ثم تشغيل حاوية اختبار بسيطة. هذا سيؤكد أن جميع المكونات الأساسية تعمل.

مثال عملي

```
docker run hello-world
```

شرح الكود

يستخدم هذا الأمر `docker run` لإنشاء حاوية جديدة وتشغيلها. الاسم `hello-world` يشير إلى صورة Docker الرسمية التي صُممت خصيصًا لاختبار التثبيت. عندما تقوم بتشغيل هذا الأمر، يقوم Docker بما يلي:

- يبحث عن صورة `hello-world` محليًا.
- إذا لم يجدها، يقوم بسحبها (pull) من Docker Hub.
- يشغل حاوية جديدة من هذه الصورة.
- تقوم الحاوية بطباعة رسالة تأكيد مثل "Hello from Docker!" ثم تتوقف.

ظهور رسالة "Hello from Docker!" يعني أن تثبيت Docker يعمل بشكل ممتاز.

نقاط رئيسية

- استخدم الأمر `docker --version` للتحقق من إصدار Docker المثبت لديك.
- أمر `docker run hello-world` هو الطريقة القياسية لاختبار وظائف Docker الأساسية.
- نجاح هذا الأمر يؤكد أن Docker قادر على سحب الصور، تشغيل الحاويات، وإخراج النتائج بشكل صحيح.

التعامل مع صور Docker

(Images) — مفهوم صور Docker ودورها

مفهوم صور Docker ودورها

تُعد صور (Docker Images) Docker اللبنة الأساسية في عالم Docker، فهي تمثل أساس بناء وتشغيل التطبيقات ضمن بيئة الحاويات. فهم الصور هو الخطوة الأولى نحو إتقان Docker.

ما هي صورة Docker؟

صورة Docker هي حزمة ثابتة، جاهزة للتنفيذ، وتحتوي على كل ما يحتاجه التطبيق للعمل: كود التطبيق، مكتبات وقت التشغيل، أدوات النظام، وملفات الإعدادات. يمكن اعتبارها بمثابة **قالب للقراءة فقط** أو مخطط (blueprint) لإنشاء حاويات Docker.

مثال عملي: عرض الصور المحلية

```
docker images
```

شرح الكود

يُستخدم الأمر `docker images` لعرض قائمة بجميع صور Docker المحفوظة على جهازك المحلي. تُظهر القائمة معلومات مثل اسم الصورة (REPOSITORY)، علامتها (TAG)، معرفها (IMAGE ID)، وتاريخ إنشائها (CREATED)، وحجمها (SIZE). هذا يساعدك على معرفة أي الصور متوفرة لديك لاستخدامها.

- صورة Docker هي قالب ثابت وغير قابل للتعديل (-read-only)، تحتوي على كل ما يحتاجه التطبيق.
- تستخدم لإنشاء حاويات Docker، حيث أن كل حاوية هي نسخة عاملة من الصورة.
- يمكن سحب الصور من مستودعات عامة (مثل Docker Hub) أو بناؤها محلياً باستخدام ملف Dockerfile.

التعامل مع صور Docker (Images) — سحب الصور من Docker Hub باستخدام الأمر `docker pull`

سحب الصور من Docker Hub باستخدام الأمر `docker pull`

تُعد صور Docker أساس الحاويات، حيث تحتوي على كل ما تحتاجه لتشغيل تطبيقك. في هذا الدرس، سنتعلم كيفية الحصول على هذه الصور الجاهزة من مستودع Docker Hub العام باستخدام الأمر `docker pull`.

المفهوم الأساسي: سحب الصور (Pulling Images)

أمر `docker pull` هو بوابتك لتحميل الصور الرسمية أو المخصصة من Docker Hub إلى جهازك المحلي. Docker Hub هو أكبر مكتبة لصور Docker، حيث يمكنك العثور على صور لمعظم أنظمة التشغيل والتطبيقات الشائعة جاهزة للاستخدام.

مثال عملي

```
docker pull ubuntu:latest
```

شرح الكود

يقوم هذا الأمر بسحب صورة نظام التشغيل "أوبونتو" (Ubuntu) بالإصدار "latest" (الأحدث) من Docker Hub إلى جهازك. بعد السحب، ستكون الصورة متاحة محليًا، مما يتيح لك استخدامها لإنشاء حاويات جديدة.

- يُستخدم الأمر `docker pull` لجلب الصور من Docker Hub.
- Docker Hub هو مستودع مركزي للصور العامة والخاصة.
- يمكنك تحديد اسم الصورة والوسم (tag) للحصول على إصدار معين منها (مثال: `ubuntu:20.04`).

التعامل مع صور Docker

(Images) — عرض الصور

المتوفرة محلياً وفحص

تفاصيلها

عرض الصور المتوفرة محلياً وفحص

تفاصيلها

في هذا الدرس، سنتعلم كيفية رؤية صور Docker المخزنة على جهازك المحلي وكيفية استكشاف تفاصيلها الدقيقة. فهم صور Docker وكيفية فحصها هو خطوة أساسية لإدارة تطبيقاتك وحاوياتك بفعالية.

عرض الصور المحلية

لعرض جميع صور Docker المتوفرة على جهازك المحلي، يمكنك استخدام أمر بسيط يعرض قائمة بجميع الصور مع معلوماتها الأساسية مثل الاسم، الإصدار (Tag)، وحجم الصورة.

فحص تفاصيل الصور

للحصول على معلومات أكثر تفصيلاً ودقة عن صورة معينة، مثل التكوينات، طبقات الصورة، ومتغيرات البيئة، يمكنك استخدام أمر

الفحص. هذا الأمر يوفر نظرة معمقة لخصائص الصورة.

مثال عملي

```
docker images
docker inspect ubuntu:latest
```

شرح الكود

السطر الأول `docker images` يعرض لك قائمة بجميع صور Docker المخزنة على جهازك المحلي، مع تفاصيل مثل اسم المستودع، الإصدار، معرف الصورة، تاريخ الإنشاء، وحجمها.

السطر الثاني `docker inspect ubuntu:latest` يقوم بتقديم معلومات تفصيلية وشاملة بصيغة JSON عن الصورة المسماة `ubuntu` ذات الإصدار `latest`، مما يساعدك على فهم تكوينها الداخلي.

- استخدم `docker images` لرؤية قائمة بجميع الصور المتوفرة على جهازك المحلي.
- استخدم `docker inspect <اسم_أو_معرف_الصورة>` لفحص تفاصيل دقيقة وشاملة لصورة محددة.
- فهم كيفية عرض الصور وفحصها يساعدك في إدارة بيئة Docker الخاصة بك وتصحيح الأخطاء.

التعامل مع صور Docker

(Images) — حذف الصور غير المستخدمة من النظام

حذف الصور غير المستخدمة من النظام

مع مرور الوقت، تتراكم صور Docker على نظامك وتستهلك مساحة قرص كبيرة. في هذا الدرس، ستتعلم كيفية تنظيف نظامك من الصور التي لم تعد قيد الاستخدام لتحرير مساحة التخزين والحفاظ على بيئة عمل مرتبة وفعالة.

لماذا نحذف صور Docker غير المستخدمة؟

قد تحتوي بيئة عملك على العديد من صور Docker التي تم سحبها أو بناؤها ولم تعد مرتبطة بأي حاويات (containers) نشطة. حذف هذه الصور يساعد على تحرير مساحة التخزين الثمينة على القرص، ويقلل من الفوضى، ويحسن من أداء إدارة Docker لديك.

مثال الكود

```
docker image prune -a
```

شرح الكود

الأمر `docker image prune` يُستخدم لإزالة الصور التي لم تعد مرتبطة بأي حاوية موجودة أو يتم الإشارة إليها بواسطة صورة أخرى. إضافة الخيار `-a` (اختصار لـ `--all`) يجعله يحذف جميع الصور غير المستخدمة، بما في ذلك تلك التي لا تزال لديها طبقات وسيطة (dangling images) والتي لا ترتبط بأي صورة عليا. هذا الأمر يقوم بتنظيف شامل للصور المهملة.

- **توفير المساحة:** إزالة الصور القديمة وغير المستخدمة يوفر مساحة كبيرة على القرص الصلب.
- **أمر فعال:** `docker image prune -a` هو الأداة الأكثر فعالية لتنظيف شامل لصور Docker.
- **صيانة دورية:** يُحسن التنظيف الدوري من إدارة صور Docker ويحافظ على نظامك مرتبًا ومنظمًا.

تشغيل وإدارة حاويات Docker

(Containers) — تشغيل حاوية Docker الأولى باستخدام الأمر `docker run`

تشغيل حاوية Docker الأولى: `docker run`

في هذا الدرس، سنتعلم كيفية تشغيل أول حاوية Docker لك باستخدام الأمر الأساسي `docker run`. هذا الأمر هو بوابتك لتجربة التطبيقات في بيئات معزولة ومحمولة.

المفهوم الأساسي: الأمر `docker run`

الأمر `docker run` هو الأداة الرئيسية لتشغيل حاوية جديدة من صورة Docker محددة. عند تشغيله، يقوم Docker بسحب الصورة (إذا لم تكن موجودة محليًا)، ثم ينشئ حاوية منها ويشغلها تلقائيًا. يمكنك تحديد خيارات متعددة لتخصيص سلوك الحاوية.

مثال عملي

```
docker run -d -p 80:80 --name my-nginx nginx
```

شرح الكود

يقوم هذا الأمر بتشغيل حاوية جديدة تقوم بتشغيل خادم الويب Nginx. لنفصل المكونات:

- `docker run`: الأمر لبدء حاوية جديدة.
- `-d`: يجعل الحاوية تعمل في الخلفية (detached mode)، مما يحرر سطر الأوامر الخاص بك.
- `-p 80:80`: يقوم بربط المنفذ 80 على جهازك المحلي بالمنفذ 80 داخل الحاوية، مما يسمح بالوصول إلى Nginx من متصفحك.
- `--name my-nginx`: يعطي اسمًا للحاوية هو "my-nginx" لسهولة الإشارة إليها لاحقًا.
- `nginx`: اسم الصورة التي سيتم استخدامها لإنشاء الحاوية.

خلاصة الدرس

- الأمر `docker run` هو الأساس لتشغيل حاويات Docker.
- يمكنك استخدام خيارات مثل `-d` للتشغيل في الخلفية و `-p` لربط المنافذ.
- كل حاوية يتم إنشاؤها تعمل بشكل مستقل ومعزول عن النظام المضيف.

تشغيل وإدارة حاويات Docker

(Containers) — دورة حياة الحاوية: الإنشاء، البدء، الإيقاف، الإزالة

دورة حياة الحاوية: الإنشاء، البدء، الإيقاف، الإزالة

تتعلم في هذا الدرس كيفية التحكم الكامل في حاويات Docker، بدءًا من إنشائها وتشغيلها، وصولاً إلى إيقافها وحذفها نهائيًا. فهم هذه الدورة أساسي لإدارة تطبيقاتك بكفاءة.

مراحل دورة حياة الحاوية

تمر حاوية Docker بأربع مراحل رئيسية: **الإنشاء (Create)** حيث يتم تجهيز الحاوية، **البدء (Start)** لتشغيلها، **الإيقاف (Stop)** لإيقاف عملها مؤقتًا، وأخيرًا **الإزالة (Remove)** لحذفها بالكامل من النظام.

مثال عملي

```
docker run --name my-nginx -d nginx
docker stop my-nginx
```

```
docker start my-nginx  
docker rm my-nginx
```

شرح الكود

الأمر **docker run** يقوم بإنشاء وتشغيل الحاوية المسماة "my-nginx" من صورة Nginx في الخلفية. ثم يقوم **docker stop** بإيقاف عمل هذه الحاوية. لإعادة تشغيلها مرة أخرى، نستخدم **docker start**. وأخيرًا، يقوم الأمر **docker rm** بحذف الحاوية "my-nginx" بشكل دائم من النظام.

- الحاوية تمر بمراحل إنشاء، تشغيل، إيقاف، وإزالة.
- يمكن تشغيل وإيقاف الحاويات مؤقتًا دون فقدان بياناتها.
- يجب إزالة الحاويات غير المستخدمة لتنظيف النظام وتحرير الموارد.

تشغيل وإدارة حاويات Docker

(Containers) — عرض الحاويات

قيد التشغيل والموقف

عرض الحاويات قيد التشغيل والموقف

في هذا الدرس، سنتعلم كيفية عرض وإدارة حاويات Docker سواء كانت تعمل حاليًا أو متوقفة. هذه القدرة أساسية لمراقبة بيئة التطوير الخاصة بك وفهم حالة جميع الحاويات.

مراقبة حالة الحاويات

لفهم ما يحدث داخل Docker، يجب أن تعرف كيف تعرض الحاويات. أمر `docker ps` يعرض الحاويات التي تعمل حاليًا فقط، بينما أمر `docker ps -a` يعرض جميع الحاويات، بما في ذلك المتوقفة منها.

مثال عملي

```
docker ps
docker ps -a
```

شرح الكود

الأمر **docker ps** يُظهر قائمة بالحاويات التي تعمل بنشاط على نظامك، مع تفاصيل مثل المعرف، الصورة المستخدمة، الأمر الذي تم تنفيذه، وقت الإنشاء، الحالة، المنافذ، والاسم. أما الأمر **docker ps -a** فيقوم بعرض جميع الحاويات التي تم إنشاؤها، سواء كانت قيد التشغيل أو متوقفة، مما يتيح لك رؤية شاملة لكل ما لديك من حاويات.

- **docker ps**: يُستخدم لعرض الحاويات النشطة (Running) فقط.

- **docker ps -a**: يُستخدم لعرض جميع الحاويات (النشطة والمتوقفة).

- مراقبة الحاويات ضرورية لإدارة موارد Docker وتحديد المشاكل المحتملة.

تشغيل وإدارة حاويات Docker (Containers) — الوصول إلى صدفة الحاوية (Container) (Shell) وتنفيذ الأوامر

الوصول إلى صدفة الحاوية (Container) (Shell) وتنفيذ الأوامر

تُعد الحاويات بيئات معزولة، ولكن غالبًا ما نحتاج إلى التفاعل معها مباشرة لتنفيذ الأوامر أو استكشاف الأخطاء وإصلاحها. يوضح هذا الدرس كيفية الدخول إلى صدفة حاوية Docker قيد التشغيل وتنفيذ الأوامر بداخلها.

مفهوم صدفة الحاوية (Container Shell)

صدفة الحاوية هي بمثابة نافذة طرفية (terminal) داخل الحاوية نفسها. تسمح لك بالوصول إلى نظام ملفات الحاوية وتشغيل الأدوات والأوامر المثبتة داخلها، مما يمنحك تحكمًا مباشرًا في بيئتها الداخلية.

مثال عملي: الدخول إلى صدفة حاوية

لنفترض أن لديك حاوية اسمها `my_app_container` قيد التشغيل وتريد الدخول إلى صدفه Bash الخاصة بها:

```
docker exec -it my_app_container bash
```

شرح الكود

يُستخدم الأمر `docker exec` لتنفيذ أمر داخل حاوية Docker قيد التشغيل.

الخيار `-i` (interactive) يسمح لك بإدخال البيانات إلى الحاوية.

الخيار `-t` (tty) يخصص لك طرفية زائفة (pseudo-TTY) تجعل التفاعل شبيهًا بالعمل على طرفية عادية.

`my_app_container` هو اسم أو معرّف الحاوية التي تريد التفاعل معها.

`bash` هو الأمر الذي سيتم تنفيذه داخل الحاوية، وفي هذه الحالة هو فتح صدفه Bash. يمكنك استخدام `sh` إذا لم تكن Bash مثبتة.

- **التفاعل المباشر:** يتيح لك `docker exec -it` التفاعل مباشرة مع حاوية قيد التشغيل.
- **أمر مرن:** يمكنك استخدام `docker exec` لتنفيذ أي أمر داخل الحاوية، وليس فقط فتح صدفه.
- **أداة هامة:** هذه الطريقة ضرورية لاستكشاف الأخطاء وإصلاحها، فحص ملفات السجل، أو إجراء تغييرات مؤقتة داخل الحاوية.

تشغيل وإدارة حاويات Docker (Containers) — ربط المنافذ (Port Mapping) للسماح بالوصول الخارجي للحاويات

ربط المنافذ (Port Mapping) للسماح بالوصول الخارجي للحاويات

تعمل حاويات Docker بشكل معزول افتراضيًا. لتمكين الوصول إلى الخدمات والتطبيقات التي تعمل داخل الحاويات (مثل خوادم الويب أو قواعد البيانات) من خارج بيئة Docker، نحتاج إلى "ربط المنافذ" الخاصة بالحاوية بمنفذ الجهاز المضيف.

المفهوم الأساسي: ربط المنافذ

ربط المنافذ (Port Mapping) هو عملية توجيه حركة البيانات من منفذ معين على الجهاز المضيف (Host Machine) إلى منفذ محدد داخل الحاوية (Container). هذا يسمح للمستخدمين الخارجيين أو التطبيقات الأخرى على نفس الجهاز المضيف بالتواصل مع الخدمات الموجودة داخل الحاوية عبر المنفذ المكشوف على الجهاز المضيف.

مثال عملي

```
docker run -d --name my-nginx-web -p 8080:80 nginx
```

شرح الكود

يشغل هذا الأمر حاوية جديدة في الخلفية (-d) ويسميتها my-nginx-web. الجزء الأهم هو -p 8080:80، الذي يوجه أي اتصال يأتي إلى المنفذ 8080 على جهازك المضيف إلى المنفذ 80 داخل حاوية Nginx. بهذه الطريقة، يمكنك الوصول إلى خادم Nginx الذي يعمل داخل الحاوية من خلال زيارة http://localhost:8080 في متصفحك.

- **أهمية الوصول:** ربط المنافذ ضروري للسماح بالوصول إلى خدمات الحاويات من خارج بيئة Docker.
- **صيغة الربط:** يتم استخدام الخيار -p متبوعًا بـ منفذ_المضيف:منفذ_الحاوية لتحديد الربط.
- **أساس التطبيقات:** فهم ربط المنافذ هو حجر الزاوية في نشر وإدارة التطبيقات المستندة إلى Docker.

تشغيل وإدارة حاويات Docker (Containers) — فحص تفاصيل الحاوية وسجلاتها

فحص تفاصيل الحاوية وسجلاتها

تعد القدرة على فحص تفاصيل وسجلات حاويات Docker أساسية لإدارة الحاويات واستكشاف الأخطاء وإصلاحها. تتيح لك هذه الأدوات فهم ما يجري داخل الحاويات وكيف تتفاعل مع بيئة Docker.

فحص تفاصيل الحاوية (docker inspect)

يُستخدم أمر `docker inspect` للحصول على معلومات مفصلة جداً حول حاوية معينة. يعرض هذا الأمر معلومات بصيغة JSON تشمل إعدادات الشبكة، وحدود الموارد، وحالة الحاوية، وغيرها الكثير.

مثال الكود (docker inspect)

```
docker inspect [container_id_or_name]
```

شرح الكود

يعرض هذا الأمر جميع تفاصيل التكوين والتشغيل لحاوية Docker المحددة بواسطة اسمها أو معرفها الفريد. يمكن أن يكون الناتج كبيراً وغنياً بالمعلومات التي تساعد في فهم الحالة الداخلية للحاوية.

فحص سجلات الحاوية (docker logs)

يتيح لك أمر `docker logs` عرض السجلات والمخرجات القياسية (`stdout` و `stderr`) التي تولدها الحاوية. هذا الأمر حيوي لمراقبة عمل التطبيق داخل الحاوية وتتبع الأحداث أو رسائل الخطأ.

مثال الكود (docker logs)

```
docker logs [container_id_or_name]
```

شرح الكود

يُظهر هذا الأمر السجلات التي ينتجها التطبيق داخل الحاوية المحددة. يمكنك استخدامه لمتابعة سلوك التطبيق، مثل رسائل البدء، والتحذيرات، ورسائل الأخطاء.

- `docker inspect`: يوفر معلومات مفصلة عن إعدادات وحالة الحاوية في صيغة JSON.
- `docker logs`: يعرض مخرجات الحاوية القياسية (`stdout/stderr`) لمراقبة سلوك التطبيق.
- فهم هذه الأوامر ضروري لاستكشاف أخطاء Docker وإدارة الحاويات بفعالية وسهولة.

بناء صور Docker مخصصة باستخدام Dockerfile — ما هو Dockerfile وكيف يعمل؟

ما هو Dockerfile وكيف يعمل؟

في رحلتنا لبناء صور Docker مخصصة، يُعد **Dockerfile** حجر الزاوية. إنه بمثابة وصفة طهي لإنشاء صورتك الخاصة، حيث يخبر Docker خطوة بخطوة كيفية تجميع الصورة وتكوينها بكل ما تحتاجه ليعمل تطبيقك بكفاءة.

Dockerfile: وصفة بناء الصور

ال Dockerfile هو ملف نصي بسيط يحتوي على مجموعة من التعليمات المتتالية التي يستخدمها Docker لبناء صورة. كل تعليمة تمثل خطوة، مثل اختيار نظام التشغيل الأساسي، نسخ الملفات، تثبيت البرامج، وتحديد كيفية تشغيل التطبيق. كل خطوة ناجحة تُنشئ طبقة (layer) جديدة في الصورة.

مثال على Dockerfile

```
FROM python:3.9-slim-buster
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

شرح الكود

هذا الـ Dockerfile يبدأ بصورة بايثون أساسية (FROM) كنقطة انطلاق. ثم ينشئ دليل عمل داخل الصورة (WORKDIR). يقوم بنسخ ملف المتطلبات فقط لتثبيت الاعتمادات (COPY و RUN)، مما يسرع عملية البناء عند التغيير. بعد ذلك، ينسخ باقي ملفات التطبيق (COPY . .). وأخيرًا، يحدد الأمر الذي سيتم تنفيذه عند تشغيل الحاوية من هذه الصورة (CMD).

- **Dockerfile** هو ملف نصي يحتوي على جميع التعليمات اللازمة لبناء صورة Docker مخصصة.
- كل تعليمة في Dockerfile تُنشئ طبقة (layer) منفصلة في الصورة، مما يعزز قابلية إعادة الاستخدام والكفاءة.
- يُمكن Dockerfile من أتمتة عملية بناء الصور بشكل كامل، مما يضمن التناسق وقابلية التكرار في بيئات التطوير والإنتاج المختلفة.

بناء صور Docker مخصصة — باستخدام Dockerfile تعليمات Dockerfile الأساسية: FROM, RUN, CMD, ENTRYPOINT, COPY, ADD

تعليمات Dockerfile الأساسية: FROM, RUN, CMD, ENTRYPOINT, COPY, ADD

يُعد Dockerfile بمثابة وصفة لإنشاء صور Docker مخصصة. كل سطر فيه يمثل تعليمة تُنفذ لبناء طبقة جديدة في الصورة. فهم هذه التعليمات الأساسية ضروري لأي مطور يتعامل مع Docker.

مفهوم Dockerfile

ملف Dockerfile هو ملف نصي بسيط يحتوي على مجموعة من التعليمات التي يقرأها Docker خطوة بخطوة لبناء صورة (Image). كل تعليمة تُنشئ طبقة جديدة، مما يجعل الصور قابلة لإعادة الاستخدام والتحسين.

مثال عملي

```
FROM python:3.9-slim-buster
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
ADD app.py .
EXPOSE 5000
ENTRYPOINT ["python"]
CMD ["app.py", "--host", "0.0.0.0"]
```

شرح الكود

يبدأ الكود بتحديد الصورة الأساسية (Python 3.9) **FROM**. ثم يتم تعيين مجلد العمل **WORKDIR** داخل الصورة إلى `app/`. تقوم تعليمة **COPY** بنسخ ملف `requirements.txt` من جهازك إلى الصورة. بعد ذلك، تقوم **RUN** بتثبيت المكتبات المطلوبة. تُستخدم **ADD** لنسخ ملف `app.py`، وهي شبيهة بـ **COPY** ولكنها تدعم أيضاً جلب الملفات من روابط URL أو فك ضغط الأرشيفات. تُعلن **EXPOSE** عن المنفذ 5000. تحدد **ENTRYPOINT** الأمر الرئيسي الذي سيتم تنفيذه (`python`)، بينما تُقدم **CMD** الحجج الافتراضية لهذا الأمر (`app.py --host 0.0.0.0`) عند تشغيل الحاوية.

خلاصة الدرس

- **Dockerfile** هو مخطط لبناء صور Docker، وكل تعليمة فيه تُنشئ طبقة.
- **FROM** تحدد الصورة الأساسية، و**RUN** تُنفذ الأوامر أثناء البناء.

- **COPY/ADD** لنسخ الملفات، و**ENTRYPOINT/CMD** لتحديد كيفية تشغيل الحاوية.

بناء صور Docker مخصصة باستخدام Dockerfile — بناء صورة Docker من Dockerfile باستخدام الأمر ``docker build``

بناء صورة Docker من Dockerfile باستخدام الأمر ``docker build``

في هذا الدرس، سنتعلم كيفية تحويل ملف **Dockerfile** الخاص بك إلى صورة Docker قابلة للتشغيل باستخدام الأمر `docker build`. هذا هو حجر الزاوية في بناء تطبيقاتك داخل بيئات Docker، حيث يمكنك تخصيص بيئة التطبيق ومكوناته.

مفهوم الأمر ``docker build``

الأمر `docker build` هو الأداة الأساسية في Docker لإنشاء الصور. يقرأ هذا الأمر التعليمات الموجودة في ملف **Dockerfile** الخاص بك وينفذها خطوة بخطوة. كل خطوة تُنشئ طبقة جديدة في الصورة، مما ينتج عنه في النهاية صورة Docker نهائية تحتوي على تطبيقك وبيئته المحددة.

مثال عملي

لنفترض أن لديك ملف `Dockerfile` وملف `index.html` في نفس المجلد:

```
# Dockerfile
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

لإنشاء الصورة من هذا الملف، استخدم الأمر التالي في سطر الأوامر:

```
docker build -t my-custom-nginx:1.0 .
```

شرح الكود

ملف الـ **Dockerfile** يحدد الخطوات: **FROM** يحدد الصورة الأساسية `(nginx:alpine)`، **WORKDIR** يضبط دليل العمل، **COPY** ينسخ ملف `index.html` من الجهاز المحلي إلى الصورة، **EXPOSE** يعلن أن الحاوية ستستمع على المنفذ 80، و**CMD** يحدد الأمر الذي سيتم تشغيله عند بدء تشغيل الحاوية. أما الأمر `docker build -t my-custom-nginx:1.0 .` فيقوم ببناء الصورة. الـ `.` (النقطة) تشير إلى أن `Dockerfile` موجود في الدليل الحالي، و**-t** يحدد اسم ووسم (tag) للصورة الناتجة، وهو هنا `my-custom-nginx:1.0`.

نقاط رئيسية

- `docker build` هو الأمر الأساسي لتحويل `Dockerfile` إلى صورة Docker.

- ال . في الأمر `docker build` يمثل سياق البناء (build context) الذي يحتوي على Dockerfile وأي ملفات أخرى ضرورية.
- يُستخدم الوسم - (tag) t لإعطاء اسم ووسم للصورة (مثل my-app:1.0) لتسهيل التعرف عليها وإدارتها.

بناء صور Docker مخصصة باستخدام Dockerfile — أفضل الممارسات لكتابة Dockerfile فعّال وآمن

أفضل الممارسات لكتابة Dockerfile فعّال وآمن

يُعد Dockerfile بمثابة المخطط الأساسي لصور Docker الخاصة بك. تعلم أفضل الممارسات يضمن لك بناء صور خفيفة، سريعة، وآمنة، مما يعزز أداء تطبيقاتك ويقلل من نقاط الضعف المحتملة.

مبادئ الكفاءة والأمان

لإنشاء Dockerfile فعّال وآمن، يجب التركيز على تقليل حجم الصورة النهائية، استخدام البناء متعدد المراحل، الاعتماد على صور أساسية موثوقة، وتطبيق مبدأ الحد الأدنى من الصلاحيات.

مثال على Dockerfile فعّال (تطبيق Go بسيط)

```
# المرحلة 1: بناء التطبيق  
FROM golang:1.20-alpine as builder  
WORKDIR /app
```

```
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /usr/local/bin/myapp

# المرحلة 2: إنشاء صورة تشغيل صغيرة
FROM alpine:latest
COPY --from=builder /usr/local/bin/myapp /usr/local/bin/myapp
EXPOSE 8080
CMD ["myapp"]
```

شرح الكود

هذا المثال يوضح **البناء متعدد المراحل (Multi-stage build)**. في المرحلة الأولى (builder)، يتم استخدام صورة Golang لبناء التطبيق وتصريفه. في المرحلة الثانية، يتم استخدام صورة Alpine صغيرة جداً، ويتم نسخ فقط الملف التنفيذي المصروف من المرحلة الأولى إليها. هذا يقلل بشكل كبير من حجم الصورة النهائية، حيث لا تتضمن أدوات البناء غير الضرورية.

- **استخدم البناء متعدد المراحل:** لتقليل حجم الصورة النهائية بشكل كبير عن طريق فصل بيئة البناء عن بيئة التشغيل.
- **قلل حجم الصورة:** تجنب تثبيت الأدوات أو الملفات غير الضرورية التي لا يحتاجها التطبيق في بيئة التشغيل النهائية.
- **استخدم صوراً أساسية رسمية وآمنة:** ابدأ من صور Docker الرسمية الموثوقة (مثل `alpine` أو صور اللغات الرسمية) التي تُحدَّث بانتظام.

تخزين البيانات في Docker

(Volumes) — فهم تحديات

استمرارية البيانات في

الحاويات

فهم تحديات استمرارية البيانات في

الحاويات

تُعد الحاويات في Docker بيئات خفيفة وسريعة، لكن لها طبيعة مؤقتة. هذا الدرس يوضح التحديات الأساسية التي تواجهنا عندما نحتاج إلى الاحتفاظ بالبيانات بشكل دائم داخل تطبيقات Docker.

المفهوم الأساسي: الحاويات مؤقتة

بشكل افتراضي، أي بيانات تُنشأ أو تُعدّل داخل الحاوية نفسها تُفقد بمجرد إيقاف الحاوية وإزالتها. هذا يعني أن التطبيقات التي تحتاج إلى تخزين معلومات مهمة (مثل قواعد البيانات، ملفات السجل، إعدادات المستخدم) لا يمكنها الاعتماد على التخزين الافتراضي للحاويات.

مثال على الكود

```
docker run --name temp-data-app -d ubuntu bash -c "echo 'This'
```

شرح الكود

يقوم هذا الأمر بتشغيل حاوية Docker باسم `temp-data-app` في الخلفية. داخل هذه الحاوية، يتم إنشاء ملف `app/data.txt/` ويكتب فيه النص "This data will disappear!". هذه البيانات موجودة الآن داخل طبقة الكتابة للحاوية. إذا قمت بإزالة هذه الحاوية لاحقًا باستخدام `docker rm -f temp-data-app`، فسُتفقد البيانات المخزنة في `app/data.txt/` بشكل دائم، مما يوضح طبيعة الحاويات المؤقتة.

نقاط رئيسية

- **طبيعة الحاويات المؤقتة:** البيانات المخزنة مباشرة داخل الحاوية تُفقد عند إزالتها.
- **الحاجة إلى استمرارية البيانات:** تتطلب التطبيقات الحقيقية (مثل قواعد البيانات) آلية للاحتفاظ ببياناتها بشكل دائم.
- **التحدي:** ضمان بقاء البيانات حتى بعد إيقاف الحاوية أو تحديثها أو إزالتها.

تخزين البيانات في Docker

(Volumes) — مفهوم الـ

Volumes وأنواعها (Bind Mounts, Docker Volumes)

مفهوم تخزين البيانات الدائم في Docker (Volumes)

في Docker، تتميز الحاويات بأنها مؤقتة بطبيعتها. هذا يعني أنه عند حذف الحاوية، تفقد جميع البيانات المخزنة بداخلها. لضمان حفظ البيانات بشكل دائم ومستقل عن دورة حياة الحاوية، نستخدم مفهوم الـ **Volumes** (وحدات التخزين).

ما هي الـ Volumes؟

الـ Volumes هي الطريقة المفضلة لتخزين البيانات الدائمة في Docker. تسمح بفصل البيانات عن الحاوية، مما يضمن بقاء البيانات حتى بعد حذف الحاوية. هناك نوعان رئيسيان:

- **Docker Volumes**: تتم إدارتها بالكامل بواسطة Docker. تُعد الخيار الأفضل لمعظم حالات الاستخدام، خاصة في بيئات الإنتاج، لسهولة إدارتها وأدائها.

- **Bind Mounts:** تربط ملفًا أو مجلدًا موجودًا على نظام المضيف (Host Machine) مباشرة داخل الحاوية. مفيدة جدًا أثناء التطوير للسماح للمطورين بتحرير الكود على المضيف ورؤية التغييرات فورًا داخل الحاوية.

مثال الكود

```
docker run -d \
  --name my_web_app \
  -p 80:80 \
  -v my_app_data:/usr/share/nginx/html \
  nginx:latest
```

شرح الكود

يقوم هذا الأمر بتشغيل حاوية Nginx جديدة باسم `my_web_app`. الجزء الهام هو `-v my_app_data:/usr/share/nginx/html`. هذا ينشئ (إذا لم يكن موجودًا) **Docker Volume** يسمى `my_app_data`، ويقوم بربطه بالمسار `/usr/share/nginx/html/` داخل الحاوية. أي ملفات تُكتب في هذا المسار داخل الحاوية ستُخزن بشكل دائم في الـ Volume `my_app_data`، حتى لو تم حذف الحاوية نفسها.

نقاط أساسية

- الـ Volumes هي الحل لتخزين البيانات الدائمة في Docker، وتفصل البيانات عن دورة حياة الحاوية.
- الـ Docker Volumes هي الخيار المفضل والمدار بواسطة Docker لمعظم التطبيقات.

- ال Bind Mounts تربط مجلدات المضيف مباشرة بالحاوية، ومفيدة لبيئات التطوير.

تخزين البيانات في Docker

(Volumes) — استخدام Bind Mounts

لربط المجلدات من المضيف

استخدام Bind Mounts لربط المجلدات من المضيف

في هذا الدرس، سنتعلم عن طريقة قوية لتخزين البيانات في Docker تُسمى "Bind Mounts". تسمح لك Bind Mounts بربط مجلد مباشر من نظام ملفات جهازك المضيف (Host) إلى مجلد داخل حاوية Docker، مما يتيح مشاركة البيانات بسهولة وفعالية.

المفهوم الأساسي: Bind Mounts

Bind Mounts هي تقنية في Docker تسمح لك بتوصيل مسار مجلد محدد على نظام ملفات جهازك المضيف (Host) مباشرة بمسار مجلد داخل حاوية Docker. هذا الربط مباشر يعني أن أي تغييرات تحدث في المجلد إما على المضيف أو داخل الحاوية ستنعكس فورًا على الطرف الآخر.

مثال عملي

```
docker run -d \
  --name my-nginx-app \
  -v "$ (pwd)/html:/usr/share/nginx/html" \
  nginx
```

شرح الكود

يُنشئ هذا الأمر حاوية Docker جديدة باسم `my-nginx-app` من صورة Nginx. الجزء الأهم هو `-v`

حيث يقوم بربط المجلد `html` الموجود في مجلد العمل الحالي على جهازك المضيف (المشار إليه بـ `$(pwd)`) بالمجلد `usr/share/nginx/html/` داخل الحاوية. هذا يعني أن ملفات الويب التي تضعها في مجلد `html` على المضيف ستكون متاحة مباشرة لـ Nginx داخل الحاوية.

- تتيح لك Bind Mounts مشاركة البيانات مباشرة بين المضيف والحاوية، مما يسهل تطوير التطبيقات.
- التغييرات في المجلد المرتبط تنعكس فورًا على الطرفين (المضيف والحاوية).
- تُستخدم Bind Mounts غالبًا لملفات التكوين، شفرة المصدر، أو البيانات المؤقتة التي تحتاج إلى الوصول إليها من المضيف.

تخزين البيانات في Docker (Volumes) — إنشاء وإدارة Docker Volumes لتخزين البيانات الدائمة

إنشاء وإدارة Docker Volumes لتخزين البيانات الدائمة

في عالم Docker، تحتاج التطبيقات غالبًا إلى تخزين البيانات بشكل دائم بحيث لا تُفقد عند حذف الحاوية أو إعادة إنشائها. هنا يأتي دور Docker Volumes كحل فعال لهذه المشكلة، حيث توفر طريقة موثوقة ومستقلة لإدارة بيانات الحاويات.

مفهوم Docker Volumes

Docker Volumes هي الطريقة المفضلة لتخزين البيانات التي تستخدمها حاويات Docker. إنها أجزاء من نظام الملفات الخاص بالمضيف، تُدار بواسطة Docker، وتُخصص لتخزين بيانات الحاويات بشكل مستقل عن دورة حياة الحاوية. هذا يضمن بقاء البيانات حتى بعد إزالة الحاوية.

مثال عملي


```
docker volume create my_data_volume
docker run -d \
  --name my_app_container \
  -v my_data_volume:/app/data \
  nginx
```

شرح الكود

يقوم الأمر الأول `docker volume create my_data_volume` بإنشاء وحدة تخزين (Volume) جديدة باسم `my_data_volume` على نظام المضيف. أما الأمر الثاني `docker run`، فيقوم بتشغيل حاوية جديدة مبنية على صورة Nginx، ويستخدم الخيار `-v` لربط وحدة التخزين التي أنشأناها `my_data_volume:/app/data` داخل الحاوية. هذا يضمن أن أي بيانات تُكتب إلى `app/data/` داخل الحاوية سيتم تخزينها بشكل دائم في `my_data_volume`.

- تُستخدم Docker Volumes لتخزين بيانات الحاويات بشكل دائم ومستقل عن الحاوية نفسها.
- يمكن ربط (Mount) نفس وحدة التخزين بأكثر من حاوية في وقت واحد.
- تُعد Volumes أكثر كفاءة وأمانًا من ربط الدلائل المباشرة (bind mounts) في معظم السيناريوهات.

تخزين البيانات في Docker

(Volumes) — أوامر إدارة الـ Volumes (إنشاء، عرض، حذف)

أوامر إدارة الـ Volumes في Docker (إنشاء، عرض، حذف)

في هذا الدرس، سنتعلم الأوامر الأساسية لإدارة وحدات تخزين البيانات في Docker، أو ما يُعرف بـ **Volumes**. هذه الأوامر تمكّنك من إنشاء وعرض وحذف الـ Volumes لضمان بقاء بيانات تطبيقاتك بشكل مستقل عن الحاويات.

ما هي الـ Docker Volumes؟

الـ Volumes هي الطريقة المفضلة لتخزين البيانات الدائمة في Docker. إنها تسمح لك بفصل البيانات عن دورة حياة الحاوية، مما يعني أن بياناتك ستبقى آمنة حتى لو تم حذف الحاوية التي تستخدمها.

أمثلة الأوامر

```
docker volume create my-app-data
docker volume ls
docker volume rm my-app-data
```

شرح الأوامر

لفهم كيفية إدارة الـ Volumes:

- الأمر الأول

```
docker volume create my-app-data
```

يقوم بإنشاء وحدة تخزين جديدة باسم

```
my-app-data
```

. هذا الاسم سيُستخدم لاحقًا لربط الوحدة بالحاويات.

- الأمر الثاني

```
docker volume ls
```

يعرض لك قائمة بجميع وحدات التخزين (Volumes) الموجودة حاليًا على نظامك، مما يتيح لك رؤية ما قمت بإنشائه.

- الأمر الثالث

```
docker volume rm my-app-data
```

يُستخدم لحذف وحدة التخزين المسماة

```
my-app-data
```

. يجب التأكد من عدم استخدام أي حاوية لهذه الوحدة قبل حذفها لتجنب فقدان البيانات.

أهم النقاط

- **أهمية ال Volumes:** تضمن استمرارية وحفظ بيانات تطبيقك بشكل دائم، بغض النظر عن الحاويات.
- **سهولة الإدارة:** توجد أوامر بسيطة وواضحة لإنشاء، وعرض، وحذف ال Volumes.
- **التحكم الكامل:** يمكنك إدارة دورة حياة بياناتك بفعالية، بدءًا من إنشائها ووصولًا إلى حذفها عند عدم الحاجة إليها.

شبكات Docker (Networking)

— نظرة عامة على شبكات Docker وأنواعها

نظرة عامة على شبكات Docker وأنواعها

تُعد شبكات Docker جزءًا أساسيًا لتمكين الحاويات (Containers) من التواصل مع بعضها البعض ومع العالم الخارجي. بدون الشبكات، لن تتمكن تطبيقاتك من العمل كفريق متكامل. هذه النظرة العامة ستعرفك على كيفية عمل شبكات Docker وأنواعها الرئيسية، وهي خطوة أولى لفهم كيفية بناء تطبيقات موزعة فعالة.

كيف تعمل شبكات Docker وأنواعها الأساسية

تُخصص Docker لكل حاوية عنوان IP خاص بها، مما يسمح لها بالتواصل بشكل سلس. هناك عدة أنواع من الشبكات المدمجة (built-in) في Docker، أبرزها: شبكة **Bridge** الافتراضية، التي تُستخدم لمعظم الحالات، و**Host** التي تدمج الحاوية مع شبكة المضيف، و**None** التي تجعل الحاوية معزولة عن الشبكة. بالإضافة إلى ذلك، يمكنك إنشاء شبكات مخصصة لتلبية احتياجات تطبيقاتك.

مثال على الكود

شرح الكود

يُظهر هذا الأمر قائمة بجميع شبكات Docker الموجودة على نظامك. ستلاحظ عادةً وجود شبكات افتراضية مثل **bridge**، وهي الشبكة الأكثر شيوعًا التي تُربط بها الحاويات بشكل تلقائي إذا لم تحدد شبكة أخرى، مما يسمح لها بالتواصل مع بعضها البعض ومع المضيف (Host).

- **شبكات Docker ضرورية:** تُمكن الحاويات من التواصل وتعمل كتطبيقات متكاملة.
- **أنواع الشبكات الرئيسية:** تشمل Bridge (الافتراضية)، Host، وNone، بالإضافة إلى إمكانية إنشاء شبكات مخصصة.
- **Bridge هي الأكثر استخدامًا:** تربط الحاويات ببعضها البعض وبالمضيف افتراضيًا.

شبكات Docker (Networking)

— شبكة الجسر الافتراضية (Default Bridge Network)

شبكة الجسر الافتراضية (Default Bridge Network)

عند تشغيل حاويات Docker بدون تحديد شبكة معينة، فإنها تتصل تلقائيًا بما يسمى "شبكة الجسر الافتراضية". هذه الشبكة هي الأساس الذي تبنى عليه معظم تطبيقات Docker البسيطة، وتوفر طريقة للحاويات للتواصل فيما بينها.

كيف تعمل شبكة الجسر الافتراضية؟

يقوم Docker بإنشاء شبكة جسر افتراضية (virtual bridge) على مضيفك (host). تتصل جميع الحاويات التي تستخدم هذه الشبكة ببعضها البعض عبر هذا الجسر، ويمكنها التواصل باستخدام عناوين IP الخاصة بها. ومع ذلك، تبقى هذه الحاويات معزولة عن الشبكة الخارجية ما لم يتم تعيين منافذ (port mapping) بشكل صريح.

مثال على الكود

```
docker run -d --name webapp1 nginx
docker run -d --name backend-service alpine sleep 3600
docker network inspect bridge
```

شرح الكود

يقوم الأمر الأول بتشغيل حاوية nginx باسم webapp1 في الخلفية، وتتصل تلقائيًا بشبكة الجسر الافتراضية. الأمر الثاني يشغل حاوية alpine باسم backend-service، وهي أيضًا تتصل افتراضيًا بنفس شبكة الجسر. الأمر الأخير يعرض معلومات تفصيلية عن شبكة الجسر الافتراضية، حيث ستلاحظ أن كلتا الحاويتين المذكورتين أعلاه مدرجتان ضمن الحاويات المتصلة بهذه الشبكة، مما يؤكد أنهما تستخدمانها افتراضيًا.

نقاط رئيسية

- هي الشبكة الافتراضية التي تتصل بها حاويات Docker عند عدم تحديد شبكة أخرى.
- تسمح للحاويات المتصلة بها بالتواصل فيما بينها باستخدام عناوين IP الداخلية.
- توفر عزلة للحاويات عن الشبكة الخارجية ما لم يتم تكوين تعيين المنافذ (port mapping).

شبكات (Networking) Docker

— إنشاء شبكات جسر مخصصة (Custom Bridge Networks)

إنشاء شبكات جسر مخصصة (Custom Bridge Networks)

في Docker، تمنحك شبكات الجسر المخصصة تحكمًا فائقًا في كيفية تواصل حاوياتك. إنها ضرورية لعزل التطبيقات وتنظيمها، مما يوفر بيئة أكثر أمانًا ومرونة مقارنة بشبكة الجسر الافتراضية.

المفهوم الأساسي

شبكة الجسر المخصصة هي شبكة افتراضية تُنشئها بنفسك داخل Docker. تسمح هذه الشبكات للحاويات المتصلة بها بالتواصل فيما بينها بسهولة باستخدام أسمائها (DNS resolution)، وتوفر عزلاً فعالاً عن الحاويات الأخرى في الشبكات المختلفة.

مثال على الكود

```
docker network create my_custom_network
docker run -d --name container1 --network my_custom_network r
docker run -d --name container2 --network my_custom_network a
```

شرح الكود

يقوم الأمر الأول `docker network create` بإنشاء شبكة جسر جديدة تحمل اسم `my_custom_network`. ثم نقوم بتشغيل حاويتين (`container1` و `container2`) ونربطهما بهذه الشبكة المخصصة باستخدام الخيار `--network`، مما يسمح لهما بالتواصل مع بعضهما البعض.

نقاط رئيسية

- **عزل الحاويات:** توفر الشبكات المخصصة بيئة معزولة لكل مجموعة من الحاويات.
- **التواصل بالاسم:** يمكن للحاويات داخل نفس الشبكة المخصصة التواصل باستخدام أسمائها بدلاً من عناوين IP.
- **مرونة التنظيم:** تسمح لك بتصميم بنية شبكة تناسب احتياجات تطبيقاتك المعقدة بشكل أفضل.

شبكات (Networking) Docker

— ربط الحاويات بالشبكات المختلفة

ربط الحاويات بالشبكات المختلفة

تعد الشبكات جزءًا أساسيًا لعمل حاويات Docker بشكل فعال. تسمح لنا هذه الشبكات بتنظيم وتأمين الاتصال بين الحاويات، أو بين الحاويات والعالم الخارجي. في هذا الدرس، سنتعلم كيف نربط حاوياتنا بشبكات مخصصة لزيادة المرونة والتحكم.

الشبكات المعرفة من قبل المستخدم

توفر شبكات Docker المعرفة من قبل المستخدم عزلًا أفضل وإمكانيات اتصال محسّنة مقارنة بالشبكة الافتراضية. تمكّن هذه الشبكات الحاويات من التواصل باستخدام أسمائها بدلًا من عناوين IP، مما يسهل إدارة التطبيقات متعددة الحاويات.

مثال على الكود

```
docker network create my-custom-network  
docker run -d --name web-server --network my-custom-network r
```

شرح الكود

الخطوة الأولى تنشئ شبكة جديدة مخصصة باسم **my-custom-network**. هذه الشبكة هي من نوع **bridge** بشكل افتراضي، وتوفر بيئة معزولة للحاويات. الخطوة الثانية تقوم بتشغيل حاوية **nginx** في الخلفية، وتسميها **web-server**، وتقوم بربطها مباشرة بالشبكة التي أنشأناها للتو.

- تُمكنك الشبكات المعرفة من قبل المستخدم من تنظيم وتأمين اتصال الحاويات بفعالية.
- يمكن ربط الحاويات بشبكات محددة عند إنشائها باستخدام الخيار **--network**.
- تتيح الشبكات المخصصة للحاويات التواصل فيما بينها باستخدام أسمائها (DNS Resolution).

شبكات (Networking) Docker — اكتشاف الحاويات (Container Discovery) داخل الشبكات

اكتشاف الحاويات (Container) (Discovery) داخل الشبكات

في هذا الدرس، سنتعلم كيف تستطيع حاويات Docker أن تجد وتتواصل مع بعضها البعض تلقائياً داخل نفس الشبكة، وهي ميزة أساسية لتطبيقات الميكروسيرفيسز وتسهيل التفاعل بين مكوناتها المختلفة.

اكتشاف الخدمات (Service Discovery) عبر DNS الدمج

توفر شبكات Docker المعرفة من قبل المستخدم خادم DNS (نظام أسماء النطاقات) مدمجاً. يسمح هذا الخادم للحاويات المتصلة بنفس الشبكة بحل أسماء الحاويات الأخرى (التي تكون عادةً أسماءها) إلى عناوين IP تلقائياً، مما يلغي الحاجة لمعرفة العناوين يدوياً.

مثال عملي

```
docker network create my-app-net
docker run -d --name web-service --network my-app-net alpine
docker run --rm --network my-app-net alpine ping -c 3 web-ser
```

شرح الكود

يقوم الكود أولاً بإنشاء شبكة مخصصة باسم `my-app-net`. بعد ذلك، يُطلق حاوية تسمى `web-service` ويربطها بهذه الشبكة. أخيراً، يُطلق حاوية أخرى (تزيل نفسها تلقائياً بعد التنفيذ) لتجرب الاتصال بـ `web-service` باستخدام اسمها فقط، مما يوضح قدرتها على اكتشافها وتحويل اسمها إلى عنوان IP داخل الشبكة.

- تستطيع الحاويات المتصلة بنفس الشبكة المعروفة من قبل المستخدم التواصل باستخدام أسمائها بكل سهولة.
- يتعامل خادم DNS المدمج في Docker مع عملية تحويل أسماء الحاويات إلى عناوين IP تلقائياً.
- يبسط هذا المفهوم بناء التطبيقات الموزعة ويجعلها أكثر مرونة وسهولة في الإدارة والتوسع.

إدارة التطبيقات متعددة الحاويات بـ Docker Compose — مقدمة إلى Docker Compose ودوره في إدارة التطبيقات

مقدمة إلى Docker Compose ودوره في إدارة التطبيقات

في هذا الدرس، سنتعرف على Docker Compose، الأداة التي تسهل إدارة التطبيقات التي تتكون من عدة حاويات Docker. سنتعلم لماذا نحتاجها وكيف تبسط عملية إعداد ونشر التطبيقات المعقدة.

مفهوم Docker Compose

Docker Compose هي أداة تمكنك من تعريف وتشغيل تطبيقات Docker متعددة الحاويات. بدلاً من تشغيل كل حاوية على حدة، تسمح لك Docker Compose بتحديد جميع خدمات تطبيقك في ملف YAML واحد، ثم إدارتها معًا كخدمة واحدة.

مثال بسيط لـ Docker Compose

هذا مثال لملف `docker-compose.yml` يقوم بتشغيل خدمة Nginx بسيطة:

```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
```

شرح الكود

يحدد هذا الملف خدمة واحدة اسمها `web`. تستخدم هذه الخدمة صورة Nginx الرسمية من Docker Hub. يقوم قسم `ports` بربط المنفذ 80 داخل حاوية Nginx بالمنفذ 80 على جهازك، مما يسمح بالوصول إلى Nginx من خلال متصفح الويب.

- **تعريف التطبيق كخدمة واحدة:** يتيح Docker Compose تحديد كل مكونات التطبيق (مثل الويب وقواعد البيانات) في ملف واحد.
- **التبسيط والإعداد السريع:** يقلل من تعقيد تشغيل عدة حاويات يدويًا، ويسرع عملية إعداد بيئات التطوير والإنتاج.
- **ملف YAML واحد:** يتم تعريف جميع الخدمات والشبكات ووحدات التخزين في ملف `docker-compose.yml` بسهولة للإدارة.

إدارة التطبيقات متعددة الحاويات بـ Docker Compose — هيكل ملف `docker-compose.yml` وتعريف الخدمات (Services)

هيكل ملف `docker-compose.yml` وتعريف الخدمات (Services)

ملف `docker-compose.yml` هو المحرك الرئيسي لتطبيقات Docker متعددة الحاويات. يحدد هذا الملف كل ما تحتاجه لتشغيل تطبيقك، من تعريف الحاويات الفردية إلى كيفية تفاعلها مع بعضها البعض. في هذا الدرس، سنتعلم هيكله الأساسي وكيفية تعريف الخدمات.

مكونات ملف `docker-compose.yml` الأساسية

يتكون الملف من أقسام رئيسية، أهمها `version` لتحديد إصدار صيغة Docker Compose، و `services` الذي يعد القلب النابض حيث تعرف كل حاوية كخدمة مستقلة ضمن تطبيقك، مع تحديد خصائصها مثل الصورة والمنفذ.

مثال على ملف `docker-compose.yml`

```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

شرح الكود

يحدد هذا المثال تطبيقاً من خدمتين. **الخدمة الأولى web** تستخدم صورة Nginx وتوجه حركة المرور من المنفذ 80 على المضيف إلى المنفذ 80 داخل الحاوية. **الخدمة الثانية db** تستخدم صورة PostgreSQL، وتُعيّن متغيرات بيئة لتهيئة قاعدة البيانات، وتستخدم مجلداً (volume) لتخزين بياناتها بشكل دائم. قسم `volumes` الأخير يحدد المجلد المستخدم.

- ملف `docker-compose.yml` يحدد البنية والتكوين لتشغيل التطبيقات متعددة الحاويات.
- قسم `services` هو المكان الذي تُعرّف فيه كل حاوية (مثل خادم الويب أو قاعدة البيانات) كخدمة.

- يمكنك تحديد الصورة (image)، المنافذ (ports)، ومتغيرات البيئة (environment) لكل خدمة بسهولة.

إدارة التطبيقات متعددة الحاويات بـ Docker Compose — تعريف الشبكات والـ Volumes في ملف Compose

تعريف الشبكات والـ Volumes في ملف Compose

تُعد إدارة التطبيقات متعددة الحاويات أكثر فعالية عند تحديد كيفية اتصال هذه الحاويات ببعضها البعض وكيفية تخزين البيانات بشكل دائم. في هذا الدرس، سنتعلم كيفية تعريف الشبكات (Networks) ووحدات التخزين (Volumes) داخل ملف Docker Compose لتحقيق ذلك.

أهمية الشبكات والـ Volumes

تسمح الشبكات للحاويات بالتواصل الآمن والمنظم فيما بينها باستخدام الأسماء بدلاً من عناوين IP، بينما تضمن الـ Volumes استمرارية البيانات، أي بقائها حتى بعد إزالة الحاوية، مما يحافظ على معلومات تطبيقك.

مثال على ملف Docker Compose

```
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - website_data:/usr/share/nginx/html
    networks:
      - app_network

volumes:
  website_data:

networks:
  app_network:
    driver: bridge
```

شرح الكود

في هذا المثال، قمنا بتعريف خدمة `web` تستخدم صورة `Nginx`. يقوم قسم `volumes` بربط وحدة التخزين المسماة `website_data` بالمسار الداخلي للحاوية `/usr/share/nginx/html/`، مما يضمن استمرارية ملفات موقع الويب. أما قسم `networks` فيربط الخدمة بالشبكة المخصصة `app_network`، مما يتيح لها التواصل مع أي خدمات أخرى مرتبطة بنفس الشبكة. في الجزء السفلي من الملف، يتم تعريف كل من `website_data` كوحدة تخزين مسماة و `app_network` كشبكة من نوع `bridge`.

- **الشبكات المخصصة:** تُمكن الحاويات من التواصل مع بعضها البعض بسهولة وآمان.
- **وحدات التخزين المسماة:** تضمن استمرارية البيانات وحفظها خارج دورة حياة الحاويات.

- **التنظيم:** يُسهل تعريف الشبكات وال Volumes في ملف Compose إدارة البنية التحتية لتطبيقك.

إدارة التطبيقات متعددة الحاويات بـ Docker Compose — تشغيل التطبيقات متعددة الحاويات باستخدام `docker-compose up`

تشغيل التطبيقات متعددة الحاويات باستخدام `docker-compose up`

يُعد الأمر `docker-compose up` أداة قوية لتشغيل وإدارة تطبيقاتك متعددة الحاويات بسهولة. يوضح هذا الدرس كيفية استخدام هذا الأمر لإطلاق جميع خدمات تطبيقك المُعرّفة في ملف `docker-compose.yml`، مما يوفر وقتًا وجهدًا كبيرين.

الأمر `docker-compose up`

هذا الأمر هو قلب Docker Compose. يقوم بقراءة ملف `docker-compose.yml` الخاص بك، وينشئ (أو يعيد بناء) الصور إذا لزم الأمر، ثم يقوم بإنشاء وتشغيل جميع الخدمات (الحاويات) والشبكات المحددة في الملف، ويجمعها معًا لتشكيل تطبيقك المتكامل.

مثال على الكود

```
# docker-compose.yml
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
  db:
    image: postgres:latest
    environment:
      POSTGRES_DB: mydatabase
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
```

بعد إنشاء الملف أعلاه، قم بتشغيل الأمر التالي في نفس المجلد:

```
docker-compose up -d
```

شرح الكود

في ملف `docker-compose.yml`، قمنا بتعريف خدمتين: `web` باستخدام صورة Nginx لتشغيل خادم الويب، و `db` باستخدام صورة PostgreSQL لقاعدة البيانات. عند تشغيل الأمر `docker-compose up -d`، سيقوم Docker Compose بقراءة هذا الملف، وإنشاء حاوية Nginx وحاوية PostgreSQL، ثم تشغيلهما معًا في الخلفية (بفضل الخيار `-d`).

- **التحكم المركزي:** يشغل `docker-compose up` جميع خدمات تطبيقك متعدد الحاويات بأمر واحد.

- **قراءة التكوين:** يعتمد الأمر على ملف `docker-compose.yml` لإنشاء وتكوين الحاويات والشبكات.
- **التشغيل في الخلفية:** استخدم الخيار `-d` (detached mode) لتشغيل الحاويات في الخلفية وتحرير طرفيتك.

إدارة التطبيقات متعددة الحاويات بـ Docker Compose — إيقاف وإزالة تطبيقات Compose

إيقاف وإزالة تطبيقات Compose

في هذا الدرس، سنتعلم كيفية إيقاف وإزالة التطبيقات متعددة الحاويات التي تديرها Docker Compose. هذا ضروري لتحرير الموارد وضمان نظافة بيئة العمل عند الانتهاء من مشروع أو الحاجة إلى إعادة البناء.

المفهوم الأساسي: إيقاف وإزالة الخدمات

عند العمل مع Docker Compose، يوجد فرق بين إيقاف الخدمات مؤقتًا وإزالتها بالكامل. إيقاف الخدمات يحافظ على حالتها، بينما إزالتها يمسح الحاويات والشبكات وحتى المجلدات التخزينية المرتبطة بها.

مثال على الكود

```
docker compose stop
docker compose down -v
```

شرح الكود

الأمر `docker compose stop` يقوم بإيقاف جميع الحاويات الجارية في تطبيق Compose بشكل مؤقت، لكنه يحافظ عليها وعلى حالتها. يمكنك إعادة تشغيلها لاحقًا بسرعة. الأمر `docker compose down -v` يقوم بإيقاف وإزالة جميع الحاويات، والشبكات، والمجلدات التخزينية (Volumes) المعرفة في ملف `docker-compose.yml`، مما يوفر تنظيفًا كاملاً للموارد.

- استخدم `docker compose stop` لإيقاف الخدمات مؤقتًا دون إزالتها.
- استخدم `docker compose down` لإزالة الحاويات والشبكات بشكل دائم.
- أضف الخيار `-v` مع `docker compose down` لإزالة المجلدات التخزينية (Volumes) المرتبطة أيضًا.

Docker Hub وسجلات الصور

(Registries) — ما هو Docker

Hub ولماذا نستخدمه؟

ما هو Docker Hub ولماذا نستخدمه؟

يُعد Docker Hub بمثابة "مكتبة" مركزية ومستودع سحابي لصور Docker، حيث يمكنك العثور على الصور الجاهزة أو تخزين ومشاركة صورك الخاصة. إنه جزء أساسي من منظومة Docker لتسهيل بناء ونشر التطبيقات.

ما هو Docker Hub؟

Docker Hub هو خدمة سحابية مجانية (مع خيارات مدفوعة) لتخزين ومشاركة صور Docker. يعمل كمستودع عام وخاص، حيث يمكن للمستخدمين سحب الصور (تحميلها) التي يحتاجونها، أو دفع صورك الخاصة (رفعها) لمشاركتها مع الآخرين أو الاحتفاظ بها بشكل آمن.

مثال عملي

```
docker pull ubuntu
```

شرح الكود

هذا الأمر يقوم بسحب (تنزيل) صورة نظام التشغيل **Ubuntu** الرسمية من **Docker Hub** إلى جهازك المحلي. بمجرد تنزيلها، يمكنك استخدام هذه الصورة لتشغيل حاويات Docker مبنية على Ubuntu.

- **مركزية التخزين:** يوفر مكانًا واحدًا لتخزين صور Docker وتنظيمها.
- **سهولة المشاركة:** يتيح للمطورين مشاركة صور تطبيقاتهم بسهولة مع فرق العمل أو المجتمع.
- **الوصول لصور جاهزة:** يوفر الآلاف من الصور الرسمية والموثوقة لأنظمة التشغيل والبرمجيات الشائعة.

Docker Hub وسجلات الصور (Registries) — إنشاء حساب على Docker Hub

إنشاء حساب على Docker Hub

مرحبًا بك في عالم Docker! Docker Hub هو خدمة أساسية تسمح لك بتخزين صور Docker ومشاركتها مع الآخرين حول العالم. لكي تتمكن من استخدام هذه الميزات القوية، تحتاج أولاً إلى إنشاء حساب خاص بك.

أهمية Docker Hub

يعمل Docker Hub كمستودع مركزي (registry) لصور Docker. يتيح لك هذا المستودع رفع صورك الخاصة (push) ومشاركتها، أو سحب صور جاهزة من الآخرين (pull)، مما يسهل التعاون وإدارة التطبيقات المعقدة.

خطوات تسجيل الدخول بعد إنشاء الحساب

بعد إنشاء حسابك بنجاح على موقع Docker Hub الرسمي (عبر المتصفح)، يمكنك استخدام الأمر التالي في سطر الأوامر لتسجيل الدخول من جهازك:

`docker login`

شرح الأمر

يُستخدم أمر `docker login` للاتصال بحسابك على Docker Hub من واجهة سطر الأوامر (CLI). عند تشغيل هذا الأمر، سيُطلب منك إدخال اسم المستخدم وكلمة المرور الخاصين بحسابك الذي أنشأته. بمجرد تسجيل الدخول بنجاح، ستتمكن من سحب الصور من Docker Hub ورفع صورك الخاصة إليه.

- **Docker Hub** هو مستودع مركزي (registry) لتخزين ومشاركة صور Docker.
- **إنشاء حساب** على Docker Hub ضروري لرفع صورك الخاصة وسحب صور الآخرين.
- أمر `docker login` يستخدم **لتسجيل الدخول** إلى حسابك من سطر الأوامر بعد إنشائه.

Docker Hub وسجلات الصور

(Registries) — وسم الصور

(Tagging Images) لرفعها

للسجل

وسم الصور (Tagging Images) لرفعها

للسجل

قبل رفع صور Docker إلى سجل خارجي مثل Docker Hub، يجب علينا وسمها بشكل صحيح. هذا الدرس يشرح كيفية وسم الصور لتحديد هويتها ومكان تخزينها في السجل، مما يجعلها جاهزة للمشاركة والاستخدام.

لماذا نوسم الصور؟

وسم الصور هو عملية إعطاء اسم وعلامة (tag) جديدين لصورة Docker موجودة محليًا. هذا الاسم الجديد عادةً ما يتضمن مسار السجل (registry path) واسم المستخدم أو المؤسسة، مما يجعل الصورة جاهزة للرفع والتمييز داخل السجل.

مثال عملي

```
docker tag my-local-app:latest your-dockerhub-username/my-local-app
```


شرح الكود

في هذا المثال، نقوم بوسم الصورة المحلية المسماة **my-local-app** وبالعلامة **latest**. الاسم الجديد يتضمن اسم المستخدم الخاص بك في Docker Hub (**your-dockerhub-username**) متبوعًا باسم المستودع (**repository**) وتصريحه (**v1.0**). هذا يجعل الصورة جاهزة للرفع إلى حسابك في Docker Hub تحت هذا الاسم الجديد.

- وسم الصور ضروري لربط الصورة بسجل معين ومستودع محدد فيه.
- صيغة الوسم تتضمن عادةً **registry- [username]/[repository-name]:[tag]**.
- باستخدام أمر **docker tag**، يمكنك إعطاء أي صورة اسمًا وعلامة جديدة، مما يسهل إدارتها ورفعها.

Docker Hub وسجلات الصور (Registries) — دفع الصور إلى Docker Hub باستخدام الأمر ``docker push``

دفع الصور إلى Docker Hub باستخدام الأمر ``docker push``

نقدم في هذا الدرس كيفية رفع صور Docker التي أنشأتها محلياً إلى Docker Hub، وهو مستودع مركزي للصور. يتيح لك Docker Hub مشاركة صورك مع الآخرين أو الوصول إليها من أي مكان في العالم.

مفهوم دفع الصور

عملية "دفع" (pushing) الصورة تعني إرسالها من جهازك المحلي إلى سجل صور (registry) مثل Docker Hub. قبل الدفع، يجب **تسمية الصورة (tagging)** بشكل صحيح لربطها بحسابك والمستودع المستهدف.

مثال عملي

```
docker tag my-app-image:latest [اسم_مستخدمك_في_Docker_Hub]/my-app:latest
docker push [اسم_مستخدمك_في_Docker_Hub]/my-app:1.0
```

شرح الكود

أولاً، يقوم الأمر **docker tag** بتسمية الصورة المحلية "my-app-image:latest" باسم جديد يشتمل على اسم المستخدم الخاص بك في Docker Hub واسم المستودع والإصدار. هذا الاسم هو ما يتعرف عليه Docker Hub. ثانياً، يقوم الأمر **docker push** برفع الصورة المسماة حديثاً إلى مستودعك في Docker Hub، مما يجعلها متاحة للجميع (إذا كان المستودع عاماً) أو لك فقط (إذا كان خاصاً).

- يستخدم الأمر **docker push** لرفع صور Docker من جهازك المحلي إلى سجلات الصور مثل Docker Hub.
- يجب تسمية (tag) الصورة بشكل صحيح بالصيغة [اسم_المستخدم]/[اسم_المستودع]:[الوسم] قبل دفعها.
- يتيح Docker Hub مشاركة صورك بسهولة مع الفرق أو الوصول إليها من خوادم مختلفة.

Docker Hub وسجلات الصور (Registries) — استخدام السجلات الخاصة (Private) (Registries)

استخدام السجلات الخاصة (Private) (Registries)

تتعلم في هذا الدرس كيف تستخدم السجلات الخاصة (Private Registries) لتخزين صور Docker بشكل آمن وخاص، بدلاً من استخدام Docker Hub العام. هذا مهم للصور التي تحتوي على معلومات حساسة أو ملكية خاصة بشركتك.

ما هي السجلات الخاصة؟

السجلات الخاصة هي مستودعات (repositories) لتخزين صور Docker تكون متاحة فقط للأشخاص أو الأنظمة المصرح لها. توفر هذه السجلات طبقة إضافية من الأمان والتحكم، مما يضمن أن صورك الحساسة لا تكون عامة.

مثال عملي

للوصول إلى سجل خاص والدفع إليه، عليك تسجيل الدخول أولاً ثم استخدام أمر الدفع:

```
docker login my.private.registry.com
docker push my.private.registry.com/my-app:v1.0
```

شرح الكود

- `docker login my.private.registry.com`: يطلب هذا الأمر منك إدخال اسم المستخدم وكلمة المرور لتسجيل الدخول إلى السجل الخاص المحدد.
- `docker push my.private.registry.com/my-app:v1.0`: بعد تسجيل الدخول بنجاح، يقوم هذا الأمر بدفع صورة Docker المسماة `my-app` بالإصدار `v1.0` إلى المستودع الخاص بك على السجل المحدد.

نقاط رئيسية

- السجلات الخاصة توفر أمانًا وتحكمًا أكبر لصور Docker.
- يجب تسجيل الدخول إلى السجل الخاص قبل سحب أو دفع الصور منه.
- اسم الصورة يجب أن يتضمن عنوان السجل الخاص عند الدفع إليه.

مفاهيم متقدمة ونصائح

عملية — البناء متعدد المراحل

(Multi-stage Builds) في Dockerfile

البناء متعدد المراحل (Multi-stage Builds) في Dockerfile

البناء متعدد المراحل هو أسلوب قوي في Dockerfile يسمح لك بإنشاء صور Docker أصغر وأكثر كفاءة عن طريق فصل بيئة البناء عن بيئة التشغيل النهائية للتطبيق.

مفهوم البناء متعدد المراحل

يعتمد هذا المفهوم على استخدام أكثر من أمر **FROM** داخل Dockerfile واحد. كل مرحلة تقوم بعمل معين (مثل بناء الكود)، وفي النهاية، يتم نسخ المخرجات المطلوبة فقط إلى الصورة النهائية النظيفة، متجاهلاً كل أدوات البناء والمكتبات غير الضرورية.

مثال عملي

```

# المرحلة الأولى: بناء التطبيق
FROM golang:1.21-alpine AS builder

WORKDIR /app
COPY . .
RUN go mod download
RUN CGO_ENABLED=0 GOOS=linux go build -o /app/my-app ./cmd/ap

# المرحلة الثانية: الصورة النهائية النظيفة
FROM alpine:latest

WORKDIR /app
COPY --from=builder /app/my-app .

CMD ["/my-app"]

```

شرح الكود

يبدأ الكود بمرحلة **builder** باستخدام صورة Golang لبناء التطبيق، حيث يتم نسخ الكود وتجميع الملف التنفيذي **my-app**. ثم تنتقل إلى المرحلة النهائية باستخدام صورة **alpine:latest** الخفيفة جدًا، وتقوم بنسخ الملف التنفيذي **my-app** فقط من المرحلة السابقة إلى هذه الصورة، مما ينتج عنه صورة Docker نهائية صغيرة جدًا تحتوي على التطبيق فقط.

- يساعد في تقليل حجم صور Docker النهائية بشكل كبير.
- يفصل أدوات البناء والمكتبات الكبيرة عن الصورة النهائية، مما يعزز الأمان.
- يبسط Dockerfile عن طريق دمج منطق البناء والتشغيل في ملف واحد بطريقة منظمة.

مفاهيم متقدمة ونصائح عملية — أساسيات أمان الحاويات وتجنب الثغرات الأمنية

أساسيات أمان الحاويات وتجنب الثغرات الأمنية

يعد أمان حاويات Docker أمرًا بالغ الأهمية لحماية تطبيقاتك وبياناتك. يتعلم هذا الدرس المبادئ الأساسية لتقليل المخاطر الأمنية وتجنب الثغرات المحتملة في بيئات الحاويات، مما يضمن تشغيل تطبيقاتك بأمان.

مبدأ الامتيازات الأقل والصور الأساسية النحيلة

يركز هذا المبدأ على منح الحاويات الحد الأدنى من الصلاحيات الضرورية لعملها، واستخدام صور أساسية صغيرة وخالية من المكونات غير الضرورية. هذا يقلل بشكل كبير من "مساحة الهجوم" المحتملة ويجعل اختراق النظام أصعب بكثير.

مثال على بناء صورة آمنة


```
# Stage 1: Build environment
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Production image
FROM alpine:3.18
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json

# Create a non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

CMD ["node", "dist/app.js"]
```

شرح الكود

يوضح هذا المثال استخدام بناء متعدد المراحل (multi-stage build) لإنشاء صورة Docker نهائية صغيرة وآمنة. في المرحلة الأولى (builder)، يتم بناء التطبيق. ثم، في المرحلة الثانية، يتم نسخ المخرجات الضرورية فقط إلى صورة **alpine** النحيلة. والأهم، يتم إنشاء مستخدم غير جذري (non-root user) باسم **appuser** وتشغيل التطبيق من خلاله، مما يحد من صلاحيات الحاوية بشكل كبير ويقلل من المخاطر الأمنية.

- **استخدم صورًا أساسية صغيرة:** اختر صورًا نحيلة مثل **alpine** لتقليل حجم الصورة ومساحة الهجوم المحتملة.
- **شغل الحاويات كمستخدمين غير جذريين:** قم بإنشاء مستخدمين غير جذريين وتشغيل التطبيقات بهم لتقليل الامتيازات الأمنية.

- **نقّذ بناءً متعدد المراحل:** استخدم multi-stage builds لضمان أن الصورة النهائية تحتوي فقط على المكونات الضرورية لتشغيل التطبيق.

مفاهيم متقدمة ونصائح عملية — تحديد حدود الموارد للحاويات (CPU, Memory)

تحديد حدود الموارد للحاويات (CPU, Memory)

في عالم Docker، من الضروري إدارة موارد الخادم بفعالية. هذا الدرس يشرح كيفية تحديد حدود استهلاك المعالج (CPU) والذاكرة (Memory) للحاويات، وهو أمر حيوي لضمان استقرار النظام ومنع استنزاف الموارد بواسطة حاوية واحدة.

المفهوم الأساسي: التحكم في استهلاك الموارد

تسمح حدود الموارد بتخصيص جزء معين من المعالج والذاكرة لكل حاوية. هذا يمنع أي حاوية من استخدام جميع موارد الخادم المتاحة، مما قد يؤثر سلبًا على أداء الحاويات الأخرى أو النظام بأكمله.

مثال عملي

```
docker run -d \  
  --name my-limited-app \  
  --cpus "0.5" \  
  ...
```

```
--memory "256m" \  
nginx
```

شرح الكود

يقوم هذا الأمر بتشغيل حاوية Nginx باسم "my-limited-app" في الخلفية. يتم تحديد استهلاكها من المعالج بحد أقصى 50% من نواة واحدة عبر `--cpus "0.5"`، والذاكرة بحد أقصى 256 ميغابايت عبر `--memory "256m"`. هذا يضمن أن الحاوية لن تتجاوز هذه الحدود المخصصة لها، مما يحمي الموارد الأخرى.

- **منع استنزاف الموارد:** تضمن حدود الموارد عدم استهلاك حاوية واحدة لجميع موارد الخادم.
- **تحسين الأداء:** تساعد في توزيع الموارد بشكل عادل بين الحاويات المختلفة على نفس المضيف.
- **زيادة الاستقرار:** تقلل من احتمالية تعطل النظام أو تدهور أدائه بسبب نقص الموارد.

مفاهيم متقدمة ونصائح عملية — تنظيف كائنات Docker غير المستخدمة (Pruning)

تنظيف كائنات Docker غير المستخدمة (Pruning)

تعد إدارة المساحة والتخلص من الكائنات القديمة أمرًا حيويًا للحفاظ على أداء Docker. سنتعلم في هذا الدرس كيفية تنظيف كائنات Docker غير المستخدمة لتحرير مساحة القرص وتحسين الكفاءة.

مفهوم "Pruning" في Docker

عملية "Pruning" (التنظيف) هي إزالة كائنات Docker التي لم تعد قيد الاستخدام، مثل الصور (Images)، والحاويات (Containers)، والشبكات (Networks)، والوحدات التخزينية (Volumes) التي لا ترتبط بأي كائنات نشطة. يساعد هذا في استعادة مساحة القرص ومنع تراكم البيانات غير الضرورية.

مثال على الكود

```
docker system prune -a
```

شرح الكود

يقوم هذا الأمر بتنظيف شامل لنظام Docker. يزيل جميع الحاويات المتوقفة، والشبكات غير المستخدمة، والصور المعلقة وغير المرتبطة بأي حاويات، بالإضافة إلى الوحدات التخزينية التي لا ترتبط بأي حاوية. الخيار `-a` أو `--all` يشمل أيضًا الصور غير المستخدمة التي لا تحتوي على علامات (tags).

- **تحرير مساحة القرص:** يساعد التنظيف في استعادة مساحات كبيرة من القرص الصلب المستخدم بواسطة Docker.
- **تحسين الأداء:** يقلل من الفوضى ويحسن من سرعة عمليات Docker مثل سحب الصور وإنشاء الحاويات.
- **أمر شامل:** `docker system prune` هو أداة قوية وفعالة لإدارة موارد Docker بشكل دوري.

مفاهيم متقدمة ونصائح عملية — استكشاف أخطاء Docker الشائعة وإصلاحها

استكشاف أخطاء Docker الشائعة وإصلاحها

عند العمل مع Docker، قد تواجه أخطاءً تمنع حاوياتك من العمل بشكل صحيح. هذا الدرس سيقدم لك إرشادات عملية لمواجهة هذه المشكلات الشائعة وكيفية تشخيصها وإصلاحها بفاعلية.

مفهوم أساسي: فهم سجلات الحاويات وحالتها

المفتاح لحل معظم مشكلات Docker يكمن في فهم سجلات الحاويات (logs) وحالتها. من خلال فحص هذه السجلات، يمكنك تحديد مصدر الخطأ، سواء كان ذلك مشكلة في التطبيق نفسه أو في بيئة Docker التي يعمل بها.

مثال عملي للكوود

```
docker run -d --name my-web-app -p 80:80 nginx:latest  
docker logs my-web-app
```

شرح الكود

يقوم الأمر الأول `docker run` بإنشاء وتشغيل حاوية Nginx باسم `my-web-app` في الخلفية، مع ربط المنفذ 80. عندما تحتاج إلى فهم سبب سلوك غير متوقع أو فشل في هذه الحاوية، يمكنك استخدام الأمر `docker logs my-web-app`. هذا الأمر يعرض جميع السجلات الصادرة من الحاوية، مما يساعدك في تشخيص المشكلة وتحديد سبب الخلل.

نقاط رئيسية

- **ابدأ بالسجلات:** دائمًا ابدأ بفحص سجلات الحاويات (`docker logs`) لتحديد رسائل الأخطاء والتحذيرات.
- **تحقق من حالة الحاويات:** استخدم `docker ps -a` للتحقق من حالة جميع الحاويات، بما في ذلك المتوقفة أو التي فشلت في التشغيل.
- **معلومات مفصلة:** استخدم `docker inspect` `<اسم_الحاوية>` للحصول على معلومات مفصلة عن إعدادات الحاوية ومواردها وإعدادات الشبكة.